

EventHub Playwright Automation Framework

Project Documentation

Author	Ravi Tanna
Role	Senior QA Engineer
Date	May 2026
Version	1.0 — Initial Release
Repository	github.com/ravitanna-jeavio/2.Playwright-Training

70	65	11	4m	0
Test Cases	Passing in CI	Modules	CI Runtime	Failures

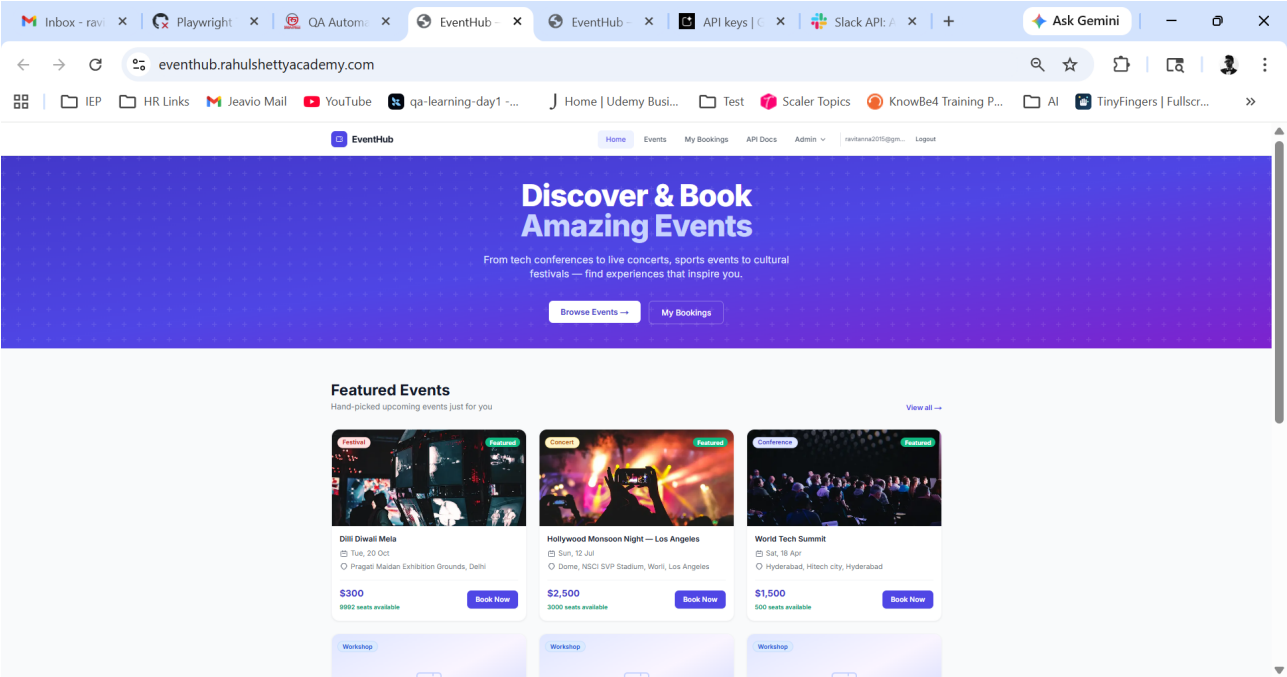


Figure 1 — EventHub application — the system under test

Table of Contents

1.	Application Under Test — EventHub	3
2.	Framework Overview	6
3.	Framework Architecture	7
4.	Self-Healing Locators	9
5.	Page Object Model	11
6.	Test Coverage & Tagging	12
7.	Data-Driven Excel Pipeline	14
8.	AI-Assisted Development + Playwright MCP	15
9.	GitHub Actions CI/CD	17
10.	How to Add New Test Cases	19
11.	Running Tests	20
12.	Tech Stack	21

SECTION 1

Application Under Test — EventHub

EventHub is a production-grade practice web application built by Rahul Shetty Academy, designed for QA engineers to build automation skills against real-world scenarios. It is a fully functional event booking platform with live REST APIs, a real database, user authentication, admin features, and a complete booking lifecycle — not a mock.

■ EventHub is a **shared sandbox environment**. Multiple QA engineers worldwide use it simultaneously. Data is real but shared — the sandbox auto-resets when booking or event limits are reached (9 bookings max, 6 custom events max per user).

Login & Authentication

Users authenticate via email and password on the login page. The framework uses a global setup step to log in once and save the browser session, so individual tests start pre-authenticated without repeating the login flow.

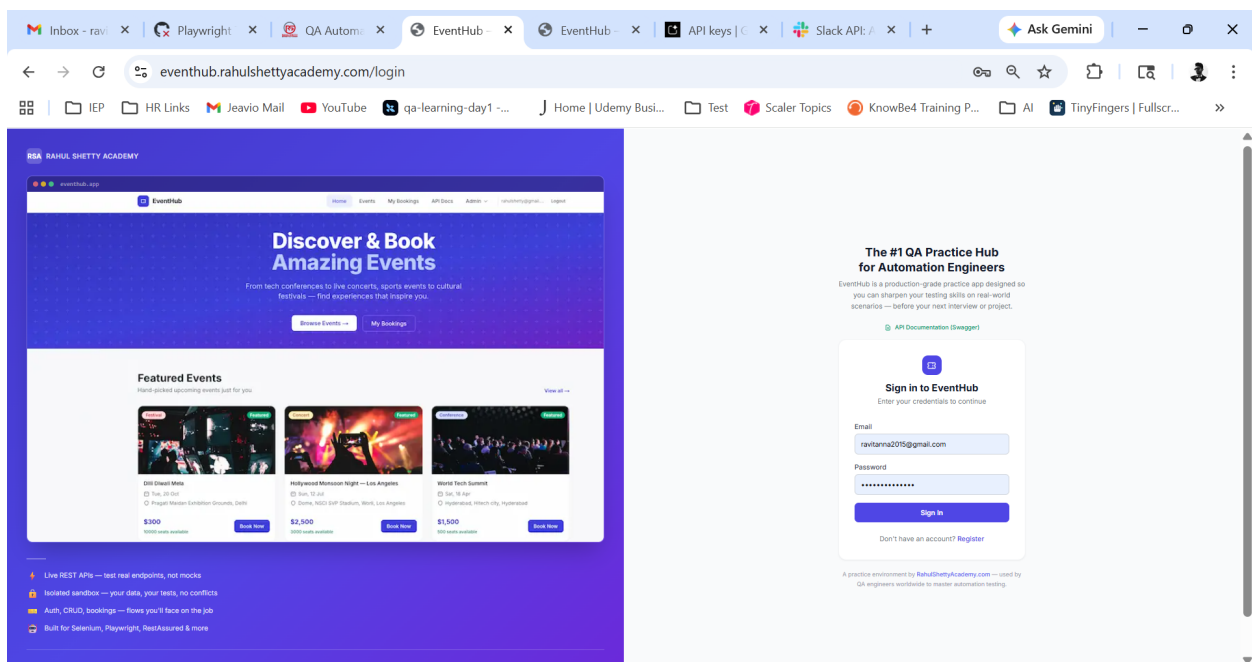


Figure 2 — Login page — credentials are managed via .env file, never hardcoded

Home Page & Featured Events

The home page displays featured events with category badges, pricing, venue, date, available seats, and a Book Now button. The navigation bar provides access to all modules including Events, My Bookings, API Docs, and the Admin panel.

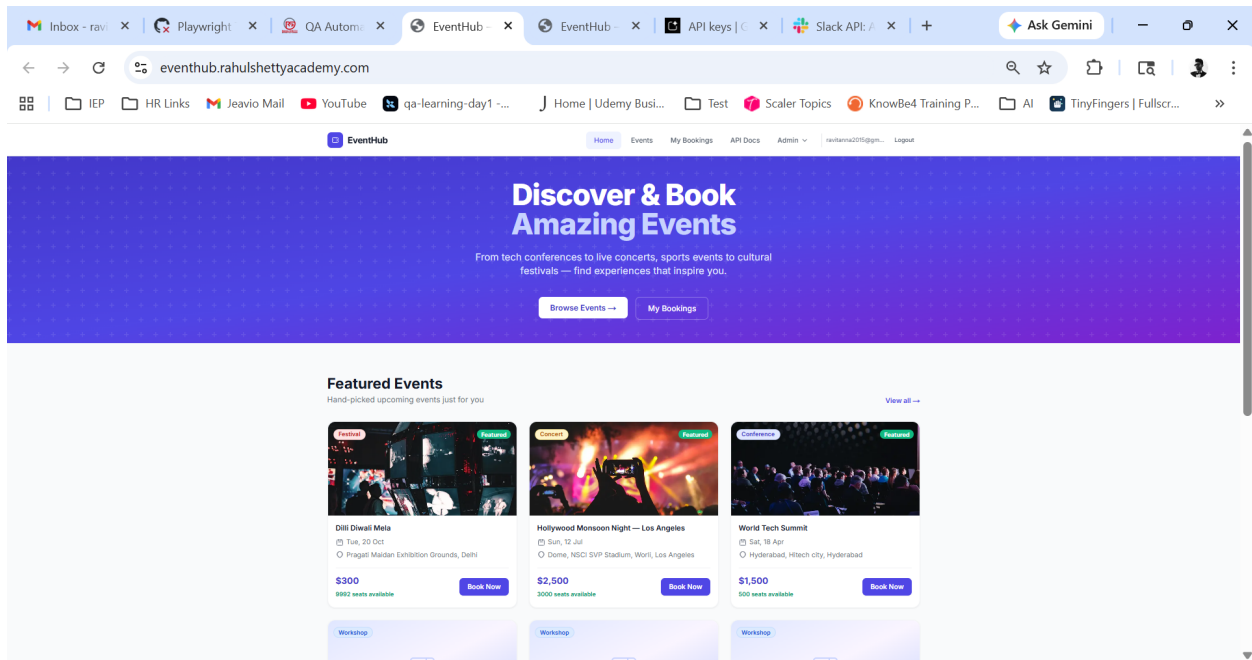


Figure 3 — EventHub Home — featured events with book now buttons

Event Listing — /events

The events listing page shows all upcoming events with search, category and city filters. Each event card displays the event image, title, date, venue, available seats and price. Note: the search input exists in the UI but returns results via filter — our tests adapted to assert card counts rather than search results.

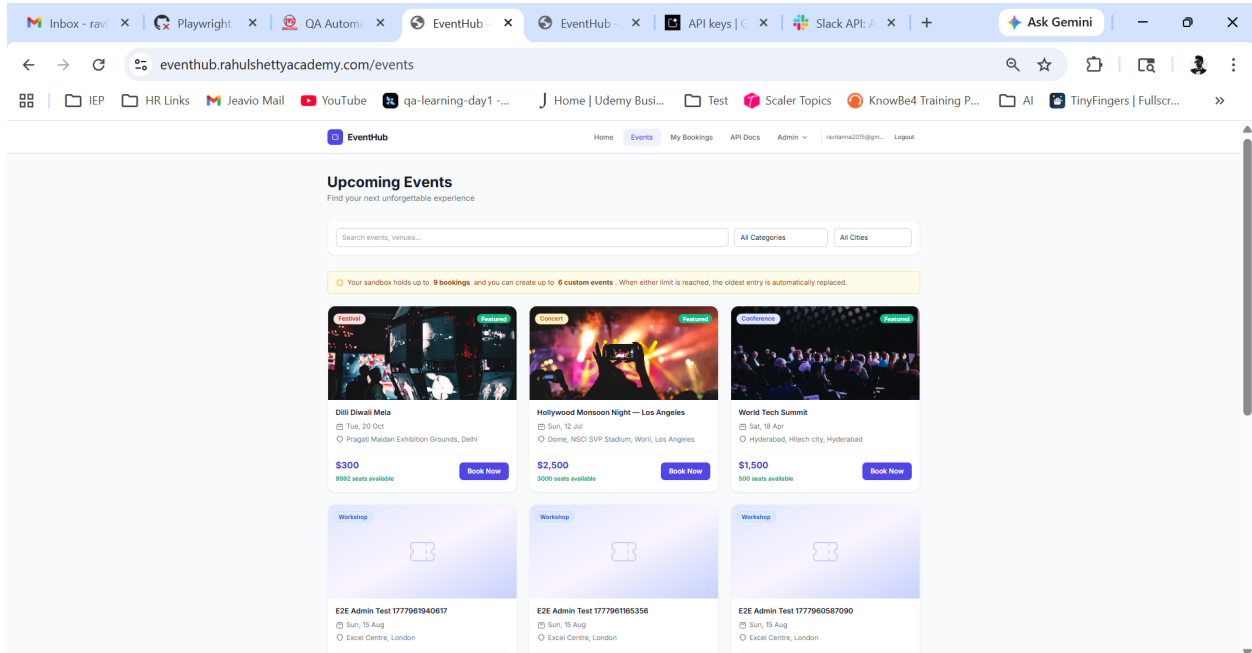


Figure 4 — Event Listing page — /events with search and filter controls

Event Detail & Booking Form — /events/:id

The event detail page shows full event information — image, title, category, date, time, venue, city, available seats and price per ticket. The booking form is inline on the same page with seat quantity selector (+/- buttons), full name, email, phone number fields, a running total, and Confirm Booking button.

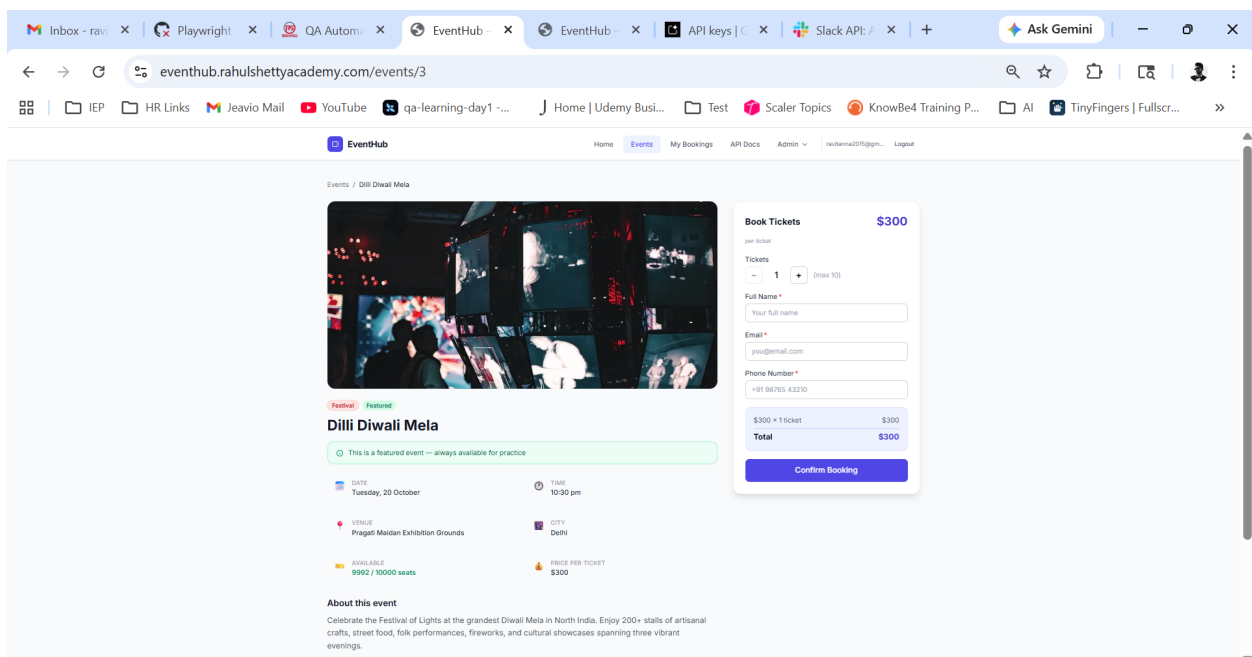


Figure 5 — Event Detail page — inline booking form with seat selector and price calculator

My Bookings — /bookings

The My Bookings page shows a user's booking history with booking reference, event name, date, ticket count, status badge, and Cancel Booking button. Each booking card links to a detail view showing full event and customer information.

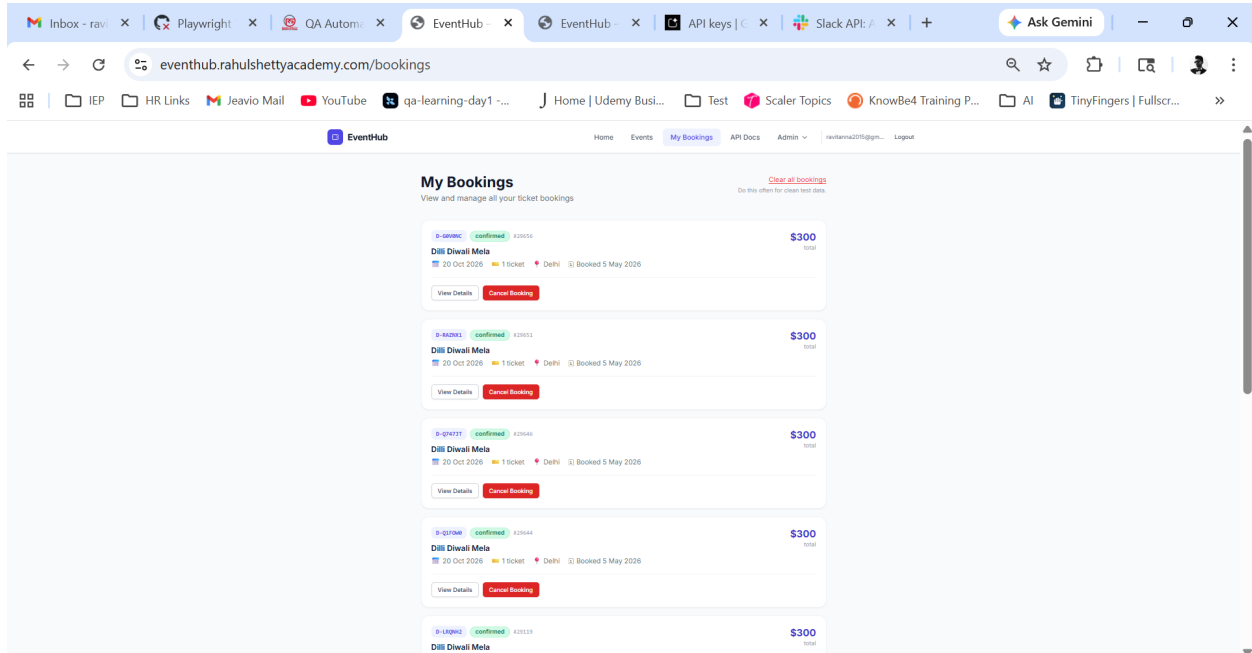


Figure 6 — My Bookings page — booking history with cancel and view details options

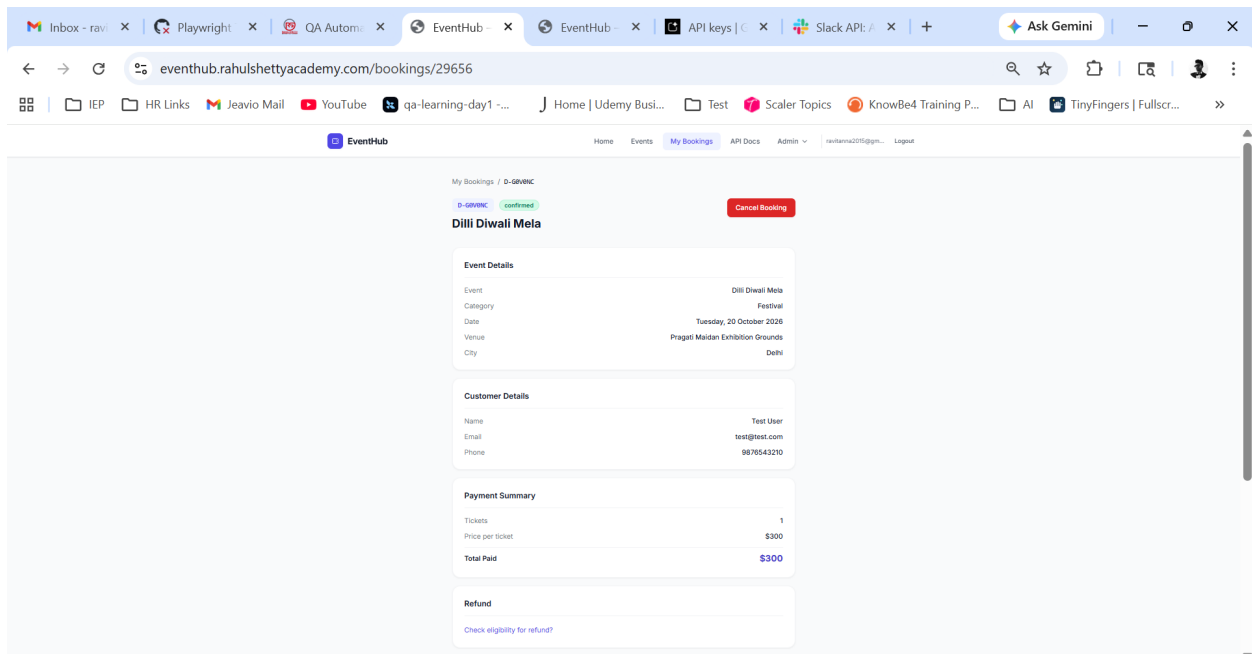


Figure 7 — Booking Detail page — event details, customer info, payment summary and refund eligibility

Admin Manage Events — /admin/events

Admin users can create new events using the form at the top of the page, entering title, description, category, city, venue, date/time, price and total seats. The All Events table below shows all events with category, city, date, price, seat availability and edit/delete actions. A limit of 6 custom events applies.

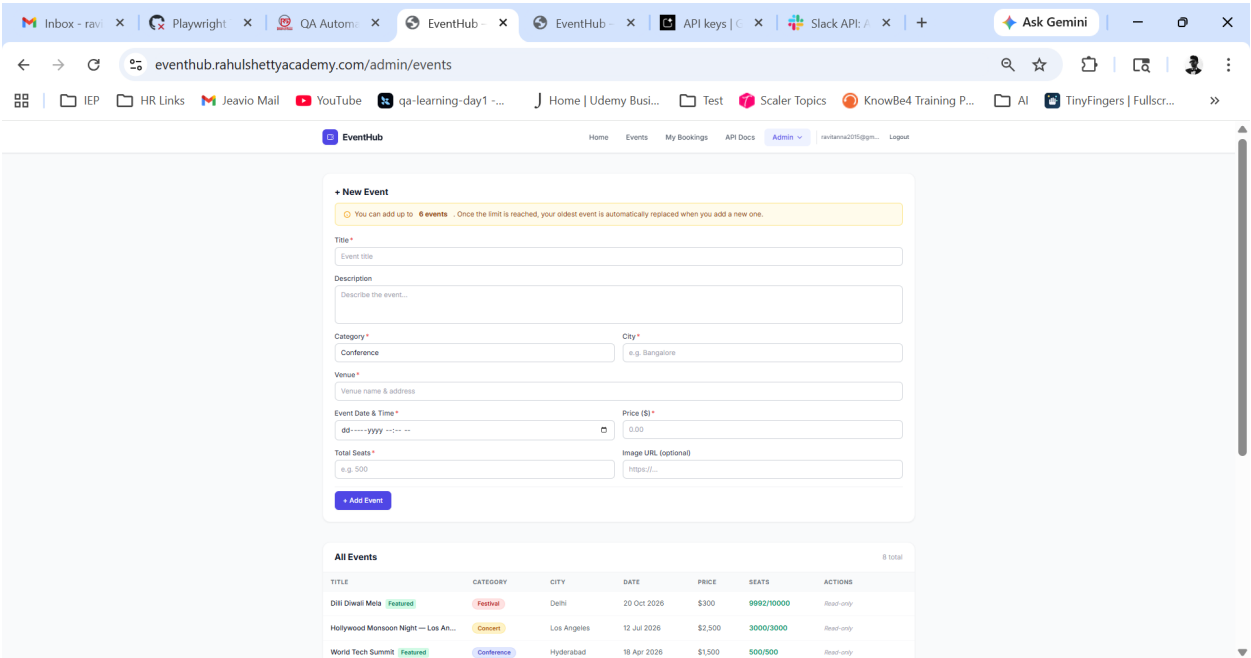


Figure 8 — Admin Manage Events — create event form and events table with actions

Admin Manage Bookings — /admin/bookings

The Admin Manage Bookings page shows all platform bookings across all users. Each row shows booking reference, customer name and email, event name, quantity, total amount, status badge, date and View/Cancel actions. A status filter allows filtering by booking status.

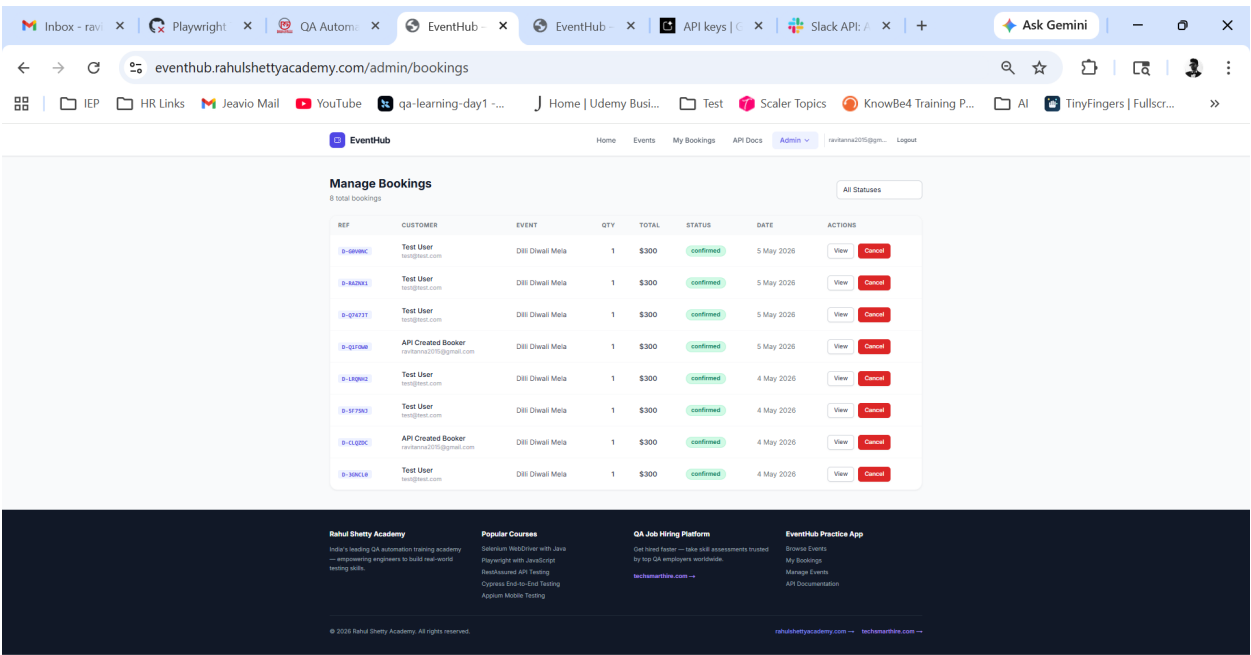


Figure 9 — Admin Manage Bookings — all platform bookings with filter and cancel actions

REST API Documentation — Swagger

EventHub exposes a complete REST API documented via Swagger / OpenAPI 3.0 at api.eventhub.rahulshettyacademy.com/api/docs. The API covers authentication (register, login, token validation), event management (full CRUD), booking operations (create, list, cancel), and user profile endpoints.

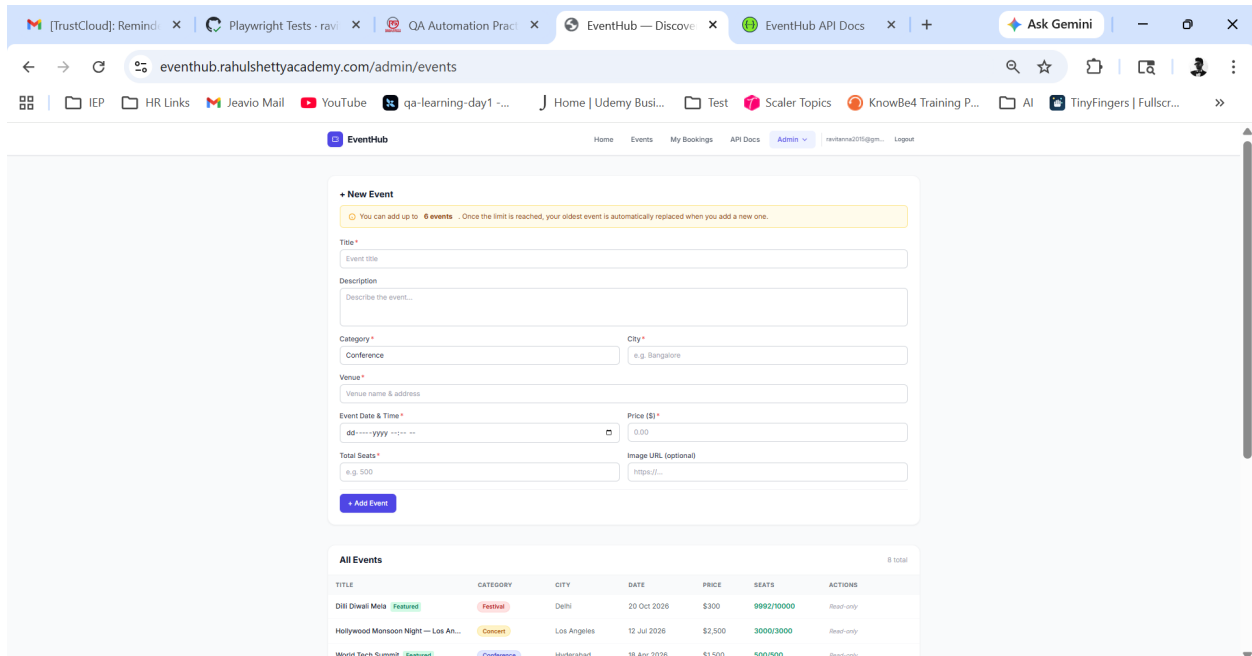


Figure 10 — EventHub Swagger API Documentation — all endpoints with request/response schemas

Application Modules Summary

Module	URL	Description	TC Range
Login / Auth	/login	Authentication — valid/invalid credentials, empty fields, security inputs	TC-001–006
Event Listing	/events	Browse events, view cards, navigate to detail page	TC-007–012
Event Detail	/events/:id	Full event info — title, description, price, venue, date, seats	TC-013–018
Event Booking	/events/:id	Book tickets — fill form, select seats, confirm booking	TC-019–024
My Bookings	/bookings	Personal booking history — view, cancel, verify status	TC-025–030
Admin Events	/admin/events	Create, edit, delete events — validation and limits	TC-031–038
Admin Bookings	/admin/bookings	All platform bookings — filter by status, cancel	TC-039–042
REST API	/api/*	Direct API testing — auth, events CRUD, bookings, errors	TC-043–066
E2E Flows	cross-module	Full end-to-end journeys crossing multiple modules	TC-067–070

SECTION 2

Framework Overview

The EventHub Playwright Automation Framework is a production-grade test automation system built from scratch over 5 days. It covers UI testing, REST API testing, and full end-to-end flows using Playwright with TypeScript. The framework is designed to be maintainable, extensible, and usable by any QA engineer without deep technical knowledge.

Key Principles

- **Single Source of Truth** — All 70 test cases maintained in one Excel file. Tests, tags and TC numbers trace back to this file.
- **No Raw Locators in Tests** — Tests never interact with pages directly. All UI interactions go through Page Object Model classes.
- **Resilience by Design** — Self-healing locators try multiple strategies before failing — tests survive UI changes automatically.
- **AI-Assisted Development** — Claude Code reads AGENT.md and SKILL.md, generates correctly structured tests from a single prompt.
- **Zero Duplication** — One Excel row = one test case = one test script. No copy-paste, no repeated setup code.
- **Full Traceability** — Every test name contains the TC number — [TC-019] — traceable from Excel to CI report.

Deliverables

Deliverable	Details
Test Scripts	70 test scripts across 9 modules — 65 active passing, 4 disabled (known issues), 6 intentional skips
Page Object Model	7 page classes — LoginPage, EventListingPage, EventDetailPage, EventBookingPage, AdminManageEventsPage, AdminManageBookingsPage, MyBookingsPage
Self-Healing Locators	SelfHealingLocator utility with priority-based fallback chains and automatic warn logging
Excel Pipeline	parseExcelTestCases() reads and validates 70 rows with Zod schema at runtime
API Testing	Full REST API coverage with Zod schema validation and auto-generated TypeScript types from Swagger
GitHub Actions CI	Scheduled daily at 7am IST, manual trigger with suite selector, HTML and JSON report artifacts
AI Integration	AGENT.md, SKILL.md, prompt-library/prompts.md — full AI-assisted development workflow

SECTION 3

Framework Architecture

The framework follows a strict 3-layer architecture. Each layer has a single responsibility and communicates only downward. Changing a locator affects only the page class — not the tests above it.

Layer 1 — Data Sources

- **test-cases-v4.xlsx** — 70 test cases across 9 module sheets — single source of truth
- **swagger.json** — EventHub OpenAPI spec — source for auto-generated TypeScript types
- **.env file** — Credentials and base URL — never committed to Git, read via `utils/env.ts`
- **Playwright MCP** — Browser context used by Claude Code to inspect live UI during development
- **playwright/.auth/user.json** — Saved auth state — generated once by `global-setup.ts`, reused by all tests

Layer 2 — Core Framework

- **utils/env.ts** — Safe `process.env` access with defaults — throws clearly if required variables missing
- **utils/constants.ts** — All shared strings, URLs and timeouts — no magic strings in the codebase
- **utils/excel-reader.ts** — `parseExcelTestCases()` — reads Excel, validates with Zod, returns typed objects
- **utils/self-healing-locator.ts** — `SelfHealingLocator` — priority-based fallback chain with warn logging
- **pages/*.ts** — 7 POM classes — all UI interactions abstracted, tests only call methods

Layer 3 — Output

- **tests/ui/*.spec.ts** — Browser-based UI tests — 7 spec files, one per module
- **tests/api/*.spec.ts** — API tests — 3 spec files covering auth, events, bookings
- **tests/e2e/*.spec.ts** — End-to-end flows crossing multiple modules
- **tests/legacy/** — Preserved learning files from Days 1–4 — not run in CI
- **GitHub Actions** — CI pipeline — HTML report + `results.json` artifact after every run

Folder Structure

2.Playwright Training/

■■■ tests/

- ■■■ ui/ ← browser tests, one file per module
- ■■■ api/ ← API tests, no browser needed
- ■■■ e2e/ ← full end-to-end flows
- ■■■ legacy/ ← preserved day* files, never run in CI
- ■■■ fixtures/ ← auth.fixture.ts, auth-state.fixture.ts
- ■■■ global-setup.ts ← saves auth state before all tests
- pages/ ← 7 Page Object Model classes
- utils/ ← constants, env, excel-reader, self-healing-locator
- test-data/ ← test-cases-v4.xlsx (source of truth)
- skills/ ← SKILL.md, prompts.md (AI configuration)
- src/api-client/ ← Api.ts (auto-generated from Swagger)
- .github/workflows/ ← playwright.yml (CI pipeline)
- AGENT.md ← AI agent briefing document
- playwright.config.ts

SECTION 4

Self-Healing Locators

Self-healing locators are the most technically advanced feature of this framework. In standard Playwright, if a developer changes a button label or renames an attribute, the test immediately fails. Self-healing locators prevent this by trying multiple ways to find an element before giving up.

Without vs With Self-Healing

■ WITHOUT Self-Healing Dev renames label → test immediately fails Engineer digs through logs Finds broken selector Manually fixes code → re-pushes Time lost: 30–60 minutes	■ WITH Self-Healing Dev renames label → Priority 1 fails silently SelfHealingLocator tries P2 getByLabel finds it → fills Logs warn: 'Falling back...' Time lost: 0 minutes ✓
---	---

Priority Order

Priority	Method	Why
1	getByRole()	Accessibility-based — most resilient to UI changes
2	getByLabel()	Form field labels — stable if semantics preserved
3	getByPlaceholder()	Input placeholders — fallback for unlabelled fields
4	getByTestId()	data-testid attributes — developer-added stable hooks
5	getByText()	Visible text content — fragile if copy changes
6	locator('css')	Last resort only — brittle, breaks with refactors

Code Example

```
this.emailInput = new SelfHealingLocator(page, [
  { name:'role', priority:1, locatorFn: p => p.getByRole('textbox',{name:/email/i}) },
  { name:'label', priority:2, locatorFn: p => p.getByLabel(/email/i) },
  { name:'placeholder', priority:3, locatorFn: p => p.getByPlaceholder(/email/i) },
  { name:'testid', priority:4, locatorFn: p => p.getByTestId('email-input') },
  { name:'css', priority:5, locatorFn: p => p.locator('input[type="email"]') },
]);

// Usage — fill() tries strategies automatically:
await this.emailInput.fill(email);
```

■ Real proof from CI logs:

[SelfHealingLocator] Falling back to strategy "button text cancel" — test continued and PASSED without any manual fix

The primary locator failed during a live CI run. The framework automatically used the fallback. No human intervention. No failure. No wasted time.

SECTION 5

Page Object Model

The Page Object Model (POM) is a design pattern where each page has a dedicated TypeScript class. The class owns all locators and all interactions for that page. Test scripts never touch selectors directly — they only call methods on page objects.

If the UI changes — say a button label changes from 'Book' to 'Reserve' — you fix it in one place (the page class) and every test using that class is automatically fixed. Without POM, you would need to find and update every test individually.

Page Class	Page URL	Key Methods
LoginPage	/login	navigate() · login() · assertRedirectedTo() · assertErrorMessage()
EventListingPage	/events	navigate() · clickFirstEventCard() · assertPageLoaded() · assertEventCountAtLeast()
EventDetailPage	/events/:id	assertAllFieldsPresent() · assertPriceVisible() · assertBookingFormVisible()
EventBookingPage	/events/:id	fillBookingForm() · book() · assertBookingConfirmed() · assertSeatDecrementDisabled()
AdminManageEventsPage	/admin/events	createEvent() · deleteEvent() · assertEventRowVisible() · clickEditEvent()
AdminManageBookingsPage	/admin/bookings	filterByStatus() · cancelFirstBooking() · assertBookingCountAtLeast()
MyBookingsPage	/bookings	navigate() · cancelFirstBooking() · assertBookingVisible() · assertEmptyState()

Test Script — POM vs Raw Playwright

```
// ■ CORRECT — using Page Object Model (what this framework does)
await loginPage.login(ENV.LOGIN_EMAIL, ENV.LOGIN_PASSWORD);
await loginPage.assertRedirectedTo('/events');

// ■ WRONG — raw Playwright in test (never done in this framework)
await page.getByLabel('Email').fill(process.env.LOGIN_EMAIL);
await page.getByLabel('Password').fill(process.env.LOGIN_PASSWORD);
await page.getByRole('button', { name: 'Sign In' }).click();
await expect(page).toHaveURL(/\/events/);
```

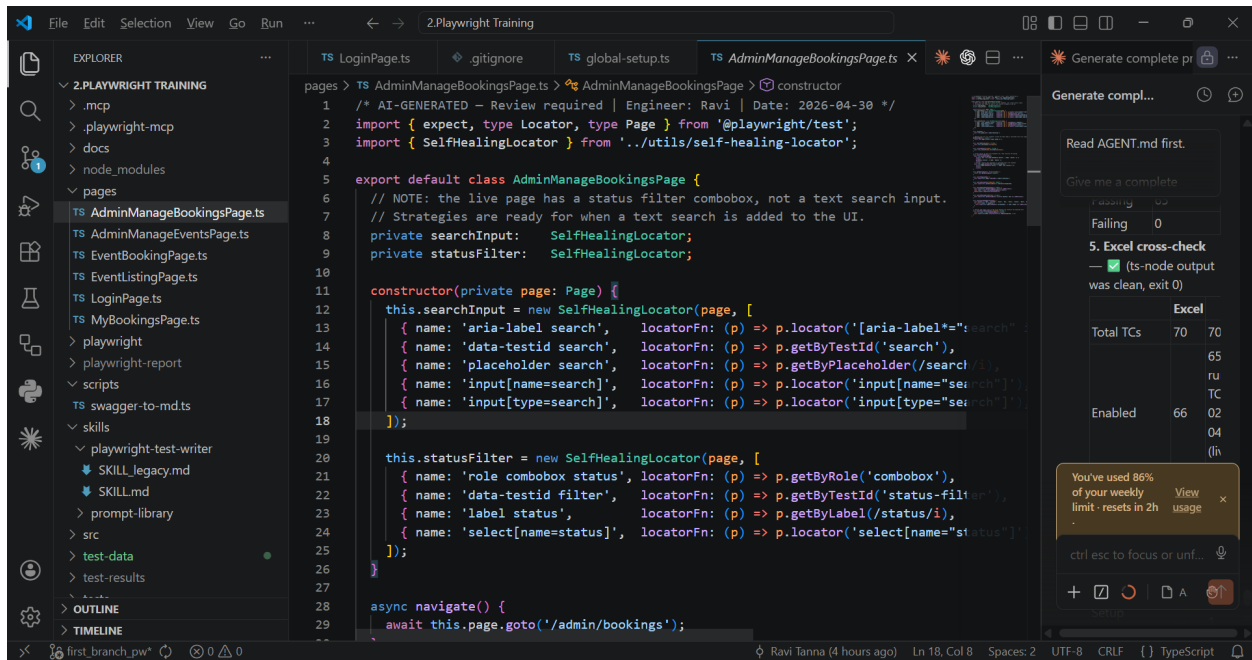
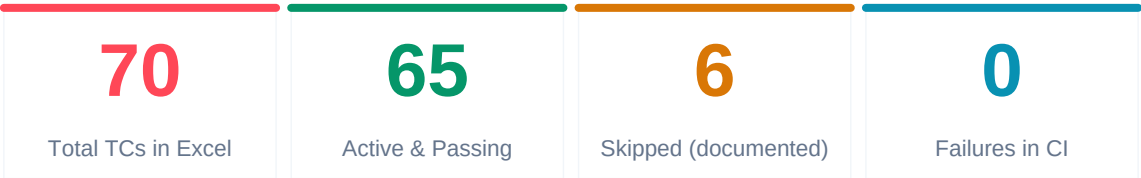



Figure 11 — EventBookingPage.ts — SelfHealingLocator chains defined in the constructor

SECTION 6

Test Coverage & Tagging

The framework provides 100% coverage of all enabled test cases in the Excel baseline. Every TC number from TC-001 to TC-070 has a corresponding test script. TC numbers are embedded in test names ensuring full traceability from Excel to CI report.



Tagging Strategy

Tag	Purpose	Approx Count	Run Time
@smoke	One critical test per module — tells you immediately if the app is alive	~7 tests	< 3 min
@regression	Major flows, happy path and edge cases — full nightly coverage	~58 tests	~ 4 min

Test Naming Convention

```
// Format: [TC-XXX] should [action] when [condition] @tag

test('[TC-001] should redirect away from login when credentials are valid @smoke', ...)
test('[TC-019] should confirm booking when all fields are filled @smoke', ...)
test('[TC-046] should return 200 and paginated array when listing events @smoke', ...)
```

Module Coverage Breakdown

Module	TC Range	Active	Skipped	@smoke	@regression	Type
Login	TC-001–006	6	0	1	5	UI
Event Listing	TC-007–012	5	1	1	4	UI
Event Detail	TC-013–018	6	0	1	5	UI
Event Booking	TC-019–024	6	0	1	5	UI
My Bookings	TC-025–030	4	2	1	3	UI
Admin Events	TC-031–038	8	0	1	7	UI
Admin Bookings	TC-039–042	3	1	1	2	UI

API	TC-043–066	22	2	3	19	API
E2E	TC-067–070	4	0	1	3	E2E
TOTAL	TC-001–070	64	6	11	53	—

SECTION 7

Data-Driven Excel Pipeline

The Excel pipeline makes the framework data-driven. A QA engineer maintains test cases in a familiar spreadsheet. The framework reads this file at runtime, validates every row with Zod, and makes data available as typed TypeScript objects. Adding a test case means adding a row to Excel — no code change needed.

Module	Sheet Name	Total Cases	High	Medium	Low	Enabled	Disabled	
Login	Login	6	4	1	1	6	0	
EventListing	EventListing	6	2	3	1	5	1	
EventDetail	EventDetail	6	2	2	2	6	0	
EventBooking	EventBooking	6	4	2	0	6	0	
BookingHistory	BookingHistory	6	3	2	1	5	1	
AdminManageEve	AdminManageEve	8	6	2	0	8	0	
AdminManageBoc	AdminManageBoc	4	2	1	1	4	0	
API	API	24	16	7	1	22	2	
E2E	E2E	4	4	0	0	4	0	
TOTALS		70	43	20	7	66	4	

Figure 12 — Excel Summary sheet — 70 test cases across 9 modules with priority and enabled status

Excel Column Schema

Column	Type	Example	Purpose
TestID	String	TC-019	Unique identifier — used in test name and --grep filter
Module	String	EventBooking	Maps to spec file and POM class
TestName	String	Confirm booking with valid data	Human-readable description
Priority	Enum	High / Medium / Low	Drives @smoke / @regression tagging
InputData	JSON String	{"name":"Ravi","seats":1}	Test data passed to POM methods
Steps	Pipe-separated	Login Navigate Fill Form Submit	Step-level description
ExpectedResult	String	Booking Confirmed heading visible	What the test asserts

TestType	Enum	UI / API / E2E	Determines which spec file
Enabled	Boolean	TRUE / FALSE	FALSE rows skipped in all runs

Parser Usage

```
// In any spec file – filter by module
const cases = parseExcelTestCases('./test-data/test-cases-v4.xlsx', {
  module: 'Login'
});

// Returns TypeScript TestCase[] – Zod validated at runtime
// Disabled rows excluded · Bad rows warned and skipped · Never crashes
```

SECTION 8

AI-Assisted Development + Playwright MCP

The framework is built to be extended by AI. Claude Code reads two configuration files at the start of every session and generates correctly structured, framework-compliant test code from a single natural language prompt. Any team member can add tests without deep Playwright expertise.

What is Playwright MCP?

Model Context Protocol (MCP) is a protocol that allows Claude Code to directly control a real browser. Instead of guessing selectors, Claude Code navigates to the live EventHub application, clicks elements, reads the actual DOM, and builds locator strategies from what genuinely exists — not assumptions. Configured in `.mcp/config.json`.

The Three AI Configuration Files

File	Purpose	When Read
AGENT.md	Full project briefing — structure, conventions, what Claude may do autonomously, output checklist	Every Claude Code session start — automatic
SKILL.md	Code generation rulebook — selector priority, POM rules, file locations, tagging rules, worked example	Referenced in prompts or session context
prompts.md	Team task library — 6 ready-made prompts for day-to-day test operations	Team member pastes relevant section into Claude Code

Workflow A — Add by TC Number

```
// Type this in Claude Code:
"Add test for TC-056"

// Claude Code will automatically:
// 1. Read test-cases-v4.xlsx and find TC-056
// 2. Identify Module, Steps, InputData, ExpectedResult
// 3. Write test in the correct spec file
// 4. Use the correct POM class for that module
// 5. Add TC number and correct @smoke/@regression tag
// 6. Run npx tsc --noEmit + playwright test --grep TC-056
```

Workflow B — Add by Module

```
// Type this in Claude Code:
"Add tests for EventBooking module"

// Claude Code will automatically:
// 1. Filter Excel for all Enabled:TRUE EventBooking rows
// 2. Write a test for each row in event-booking.spec.ts
// 3. Use EventBookingPage POM for all interactions
// 4. Tag correctly based on Priority column
// 5. Verify the full file passes before finishing
```

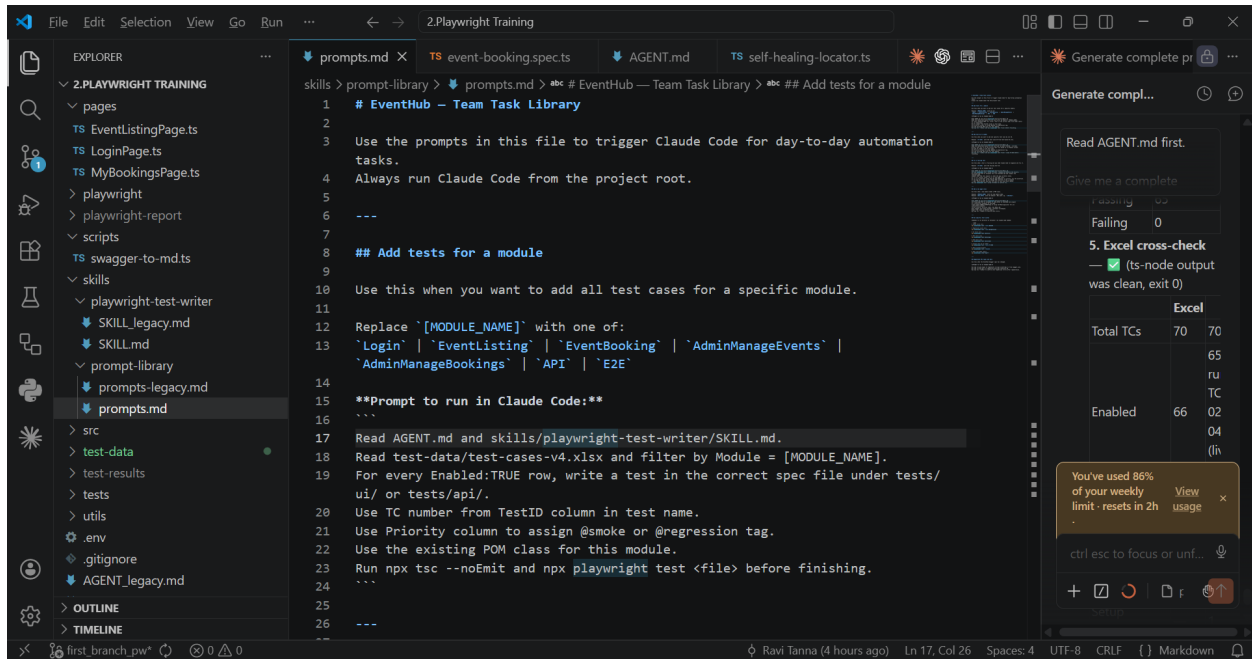


Figure 13 — prompt-library/prompts.md — team task library viewed in VS Code

SECTION 9

GitHub Actions CI/CD

The framework runs automatically via GitHub Actions — a free cloud CI service. Every weekday at 7am IST, a fresh cloud machine runs all 65 active tests against the live EventHub application and uploads the HTML report as a downloadable artifact. Email notifications are sent automatically when any run fails.

Trigger Strategy

Trigger	When	What Runs
Schedule	7am IST (01:30 UTC) Mon–Fri automatically	Full suite: tests/ui/ + tests/api/ + tests/e2e/
workflow_dispatch	Manual button in GitHub Actions tab	Suite selector: all / smoke / regression / ui / api / e2e

What Happens Every Run

1. GitHub spins up a fresh Ubuntu machine — clean slate every run, nothing from previous run persists
2. Checks out code from the main branch
3. npm ci installs all packages (npm cache used for speed — typically 4 seconds)
4. npx playwright install chromium installs the browser
5. global-setup.ts runs — logs in and saves auth state to playwright/.auth/user.json
6. 65 tests run in parallel using 2 workers — takes approximately 3–4 minutes
7. HTML report and results.json uploaded as artifacts with 30 day retention
8. Email notification sent automatically if any test fails
9. Machine is destroyed — no state carries over to the next run

GitHub Secrets

Credentials are never stored in code. They are stored in GitHub's encrypted secrets vault and injected as environment variables at runtime:

Secret	Value	Used By
BASE_URL	https://eventhub.rahulshettyacademy.com	All tests — base URL for navigation and API calls
LOGIN_EMAIL	ravitanna2015@gmail.com	Global setup + UI tests requiring authentication
LOGIN_PASSWORD	Stored securely — not visible after saving	Global setup + UI tests requiring authentication

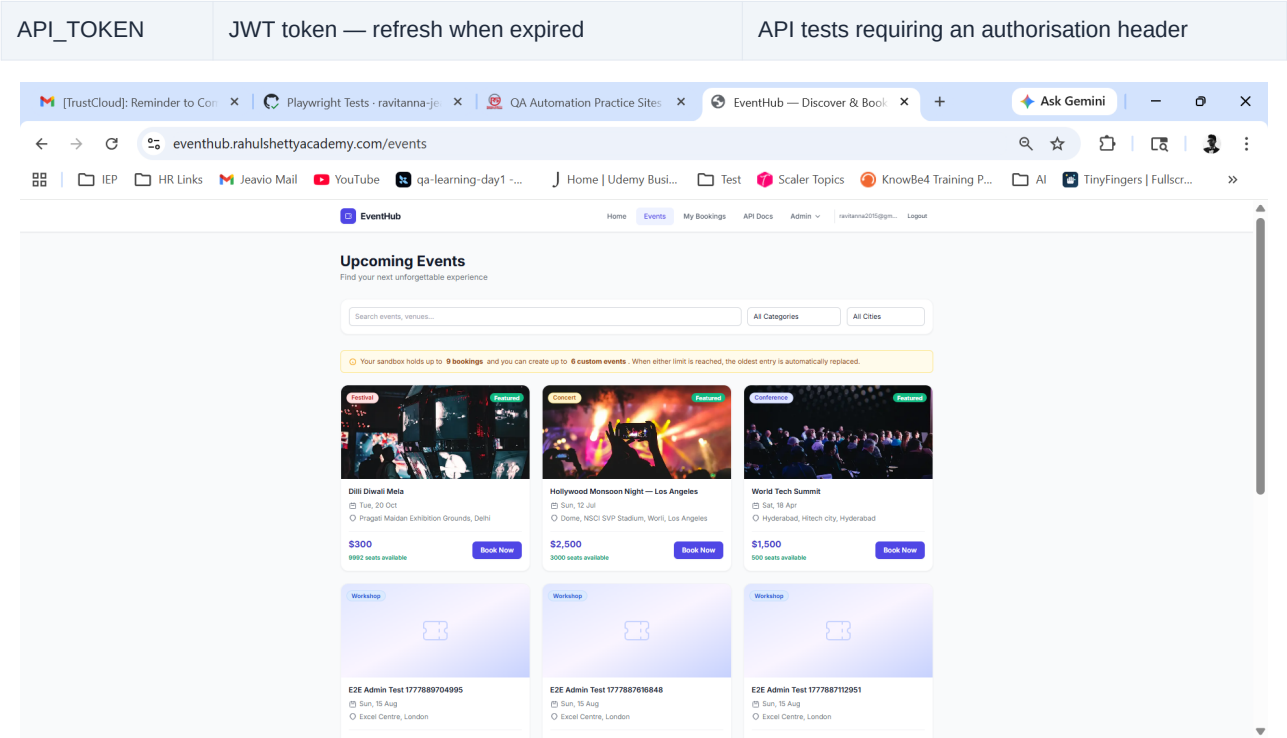


Figure 14 — GitHub Actions — successful CI run showing all steps green

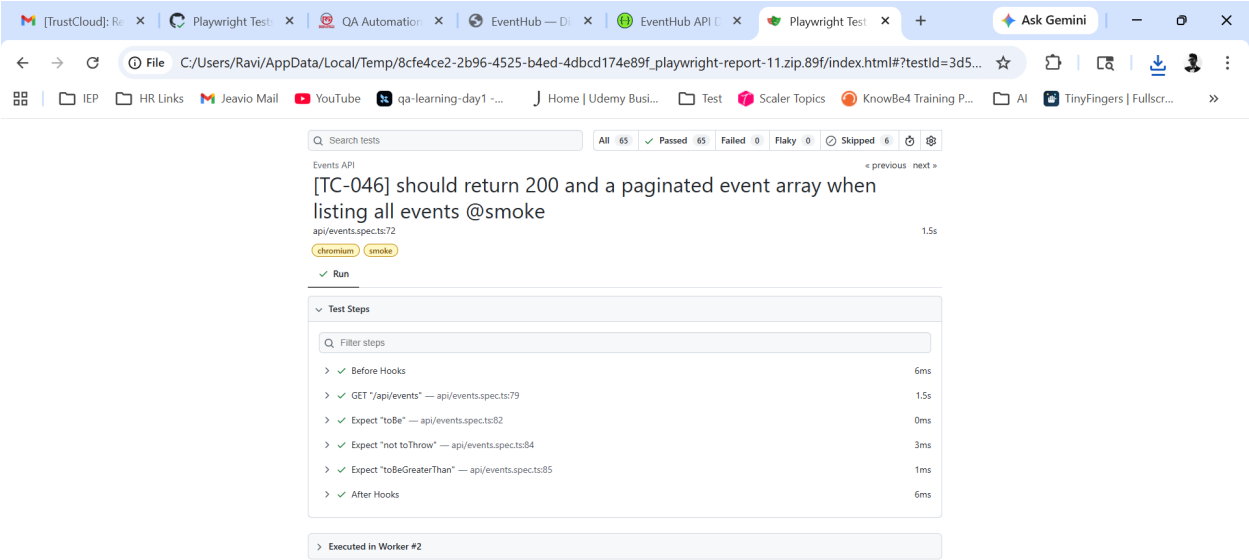


Figure 15 — Playwright HTML Report — 65 passed, 6 skipped, 0 failed, 3.1 minute runtime

SECTION 10

How to Add New Test Cases

Adding a new test case is a two-step process: add the row to Excel, then use a prompt to generate the test script. Any team member can do this.

Step 1 — Add Row to Excel

Open test-data/test-cases-v4.xlsx, navigate to the correct module sheet, and add a new row. Fill all columns: TestID (next sequential TC number), Module, TestName, Priority, InputData (JSON), Steps, ExpectedResult, TestType, Enabled.

Step 2 — Generate Test Script

Open Claude Code in VS Code and use one of the two workflows from prompt-library/prompts.md:

Workflow	When to use	Prompt to type
A — By TC Number	You know the exact TC ID	"Add test for TC-071"
B — By Module	You want all tests for a module	"Add tests for Login module"

What Claude Code Guarantees

- ✓ TC number in the test name — [TC-XXX] format matching the Excel row
- ✓ Correct @smoke or @regression tag based on the Priority column
- ✓ Test placed in the correct spec file for that module
- ✓ POM methods used — no raw locators in the test file
- ✓ AAA structure — Arrange / Act / Assert with blank lines
- ✓ Screenshot on the final assertion step
- ✓ TypeScript compiles clean — npx tsc --noEmit exits with 0 errors
- ✓ Test passes — npx playwright test --grep TC-XXX exits with code 0

SECTION 11

Running Tests

■■ Run **npm run setup** once after cloning. This logs in and saves the session to playwright/.auth/user.json. Without this step, all UI tests will fail immediately because they expect to start pre-authenticated.

Command	What It Runs
npm run setup	Runs global-setup.ts — logs in and saves auth state (run once after clone)
npm run test:smoke	Runs only @smoke tagged tests — approximately 7 tests, under 3 minutes
npm run test:regression	Runs only @regression tagged tests — full nightly coverage
npm run test:ui	Runs all tests in tests/ui/ — browser tests only
npm run test:api	Runs all tests in tests/api/ — no browser, very fast
npm run test:e2e	Runs all tests in tests/e2e/ — end-to-end flows
npm run test:all	Runs tests/ui/ + tests/api/ + tests/e2e/ — full active suite
npm run test:headed	Runs with visible browser — useful for debugging
npm run report	Opens last HTML report in browser
npx playwright test --grep TC-019	Runs one specific test by TC number
npx playwright test tests/ui/ --grep TC-019	Runs TC-019 from ui folder only (avoids legacy)

Getting Started — New Team Member

```
# 1. Clone the repository
git clone https://github.com/ravitanna-jeavio/2.Playwright-Training.git

# 2. Install dependencies
npm ci

# 3. Install Playwright browsers
npx playwright install --with-deps

# 4. Copy environment file and fill in credentials
cp .env.example .env
# Edit .env — add BASE_URL, LOGIN_EMAIL, LOGIN_PASSWORD, API_TOKEN

# 5. Save auth state (run once, re-run if session expires)
npm run setup

# 6. Verify everything works
npm run test:smoke
```

SECTION 12

Tech Stack

Every tool in the stack is production-grade, actively maintained, and battle-tested. Nothing experimental was used.

Tool	Version	Role in Framework
Playwright	Latest	Browser automation, API testing, parallel execution, trace viewer, screenshot and video capture
TypeScript	5.x	Type safety across the entire codebase — compile-time error detection, interfaces, auto-complete
Zod	3.x	Runtime schema validation for Excel rows and API responses — catches data shape errors at test time
Claude Code + MCP	Claude Sonnet	AI assistant — reads AGENT.md and SKILL.md, navigates live browser via MCP, generates compliant tests
Excel / XLSX	—	70 test cases across 9 module sheets — maintained by QA, non-technical team can contribute
openapi-typescript	Latest	Auto-generates Api.ts TypeScript types from EventHub Swagger spec — stays in sync with the API
GitHub Actions	Free tier	Cloud CI — scheduled daily, manual trigger with suite selector, artifact upload, email on failure

Framework Summary

The EventHub Playwright Automation Framework delivers production-grade test coverage across UI, API, and E2E layers. With 65 active tests passing in CI, self-healing locators that survive UI changes automatically, AI-assisted development that generates compliant tests from a single prompt, and a scheduled CI pipeline running every weekday at 7am IST — this framework operates with minimal daily human intervention.

Built by Ravi Tanna | Senior QA Engineer | May 2026
github.com/ravitanna-jeavio/2.Playwright-Training